

TP3

Avant de commencer, nous devons importer les modules nécessaires afin d'utiliser les fonctions spéciales de numpy, soit les modules suivants :

```
import numpy as np
import matplotlib.pyplot as plt
```

Grâce à cela, nous sommes dans la capacité de faire certains types d'opération :

- Définir un polynôme, nous choisissons le degré et les coefficients.
- Définir le polynôme de degré n d'inconnue x dont nous choisissons les racines.
- Effectuer des opérations usuelles sur ces polynômes.

Voici quelques exemples :

```
[2]: Q=np.poly1d([5,4,3,2,1,0])
      print(Q)
```

$$5x^5 + 4x^4 + 3x^3 + 2x^2 + 1x$$

```
[3]: P=np.poly1d(np.poly([1,2,3,4,5]))
      print(P)
```

$$1x^5 - 15x^4 + 85x^3 - 225x^2 + 274x - 120$$

```
[4]: R=P*np.poly1d([1,0])
      print(R)
```

$$1x^6 - 15x^5 + 85x^4 - 225x^3 + 274x^2 - 120x$$

```
[5]: P_prime=np.polyder(P,3)
      print(P_prime)
```

$$60x^2 - 360x + 510$$

```
[6]: print(np.polyval(Q,2))
```

258

```
[7]: print(P+Q)
```

$$6x^5 - 11x^4 + 88x^3 - 223x^2 + 275x - 120$$

Dans la question suivante, il nous est demandé d'écrire une fonction python Lagrange(A,k) qui prend en argument le vecteur $A = [a_0, \dots, a_n]$ des points d'interpolation et un entier k et renvoie le k-ième polynôme élémentaire de Lagrange :

Pour se faire, nous mettons ce que nous avons appris en cours sous forme d'algorithme :

```
def lagrange(A,k):
    p=1
    n=len(A)-1
    for i in range(n):
        if k!=A[i]:
            p=p*(np.poly1d([1,0])-A[i])/(A[k]-A[i])
    print(p)
```

Pour se faire, on initialise le polynôme élémentaire à la valeur 1, car nous le multiplierons plusieurs fois dans une boucle for, dont le nombre d'itération dépend de la longueur de A, le vecteur choisit. Il nous reste seulement à appliquer la formule vue en cours à l'intérieur de notre boucle, en faisant attention à ce que la valeur de l'entier k soit différente des points du vecteur A.

Lorsque nous testons la fonction ainsi créée en construisant les polynômes d'interpolation élémentaire en [0,1,2], on obtient le résultat suivant :

```
lagrange([0,1,2],2)
```

```
2
0.5 x - 0.5 x
```

Ensuite, nous devons coder une fonction `interpolation(A,F)` qui prend en argument deux vecteurs de même taille et qui renvoie le polynôme d'interpolation qui prend les valeurs de F en les points de A.

On commence par initialiser le polynôme d'interpolation P à la valeur 0, car nous l'ajouterons plusieurs fois et changera constamment jusqu'à ce qu'il obtienne sa valeur finale. Dans cet algorithme, contrairement au précédent, deux boucles sont créées, car on évalue le polynôme élémentaire de Lagrange en plusieurs points. Nous pouvons ensuite appliquer la même chose que précédemment en multipliant le polynôme élémentaire au point i par la valeur de F au point i. A la fin de cette boucle, nous obtenons la valeur finale de P, notre polynôme interpolateur.

```
def interpolation(A,F):
    P=0
    n=len(A)-1
    for i in range(n):
        pi=1
        for j in range(n):
            if i!=j:
                pi=pi*(np.poly1d([1,0])-A[j])/(A[i]-A[j])
        P+=F[i]*pi
    return P
```

Dans la question qui suit, il nous est donnée une fonction f défini par :

$$f(x) = \frac{1}{1 + 25x^2}.$$

Il faut construire les polynômes d'interpolation de degré 5,10 et 20 en les points équidistants :

$$a_k = -1 + \frac{k}{n}, \quad k = 0, \dots, 2n.$$

On commence par définir notre fonction, qui prendra une valeur x et retourne la valeur de la fonction à ce point :

```
def f(x): return 1/(1+25*x**2)
```

On définit la suite a_k sous forme de vecteur, ainsi nous avons nos vecteurs A5, A10, A20:

```
def suite_ak(n):  
    return np.array([-1 + k / n for k in range(2 * n + 1)])  
  
A5=suite_ak(5)  
A10=suite_ak(10)  
A20=suite_ak(20)
```

On revient à notre fonction f pour obtenir sa valeur en fonction de ces vecteurs :

```
F5=f(A5)  
F10=f(A10)  
F20=f(A20)
```

On définit ensuite une fonction interpolation2 qui prendra une troisième variable en argument, le point x :

```
def interpolation2(A, F, x):  
    P = 0  
    n = len(A)  
    for i in range(n):  
        pi = 1  
        for j in range(n):  
            if i != j:  
                pi *= (x - A[j]) / (A[i] - A[j])  
        P += F[i] * pi  
    return P
```

Cela nous servira à facilement intégrer les polynômes d'interpolation à notre graphe.

On en profite pour définir les fonctions P5, P10 et P20 qui calculent donc le polynôme d'interpolation à n'importe quel point de x :

```
def P5(x): return interpolation2(A5,F5,x)  
def P10(x): return interpolation2(A10,F10,x)  
def P20(x): return interpolation2(A20,F20,x)
```

Pour afficher sur un graphe les courbes représentatives de f et des polynômes interpolateurs de Lagrange de degrés 5, 10 et 20, nous devons savoir de quoi composées nos coordonnées X et Y .

On définit l'axe X qui contiendra un grand nombre de valeurs entre 0 et 1, et Y_5 , Y_{10} , Y_{20} qui correspondent simplement à leur polynôme interpolateur en fonction des valeurs de X :

```
X=np.linspace(-1,1,1000)
```

```
Y5 = P5(X)
```

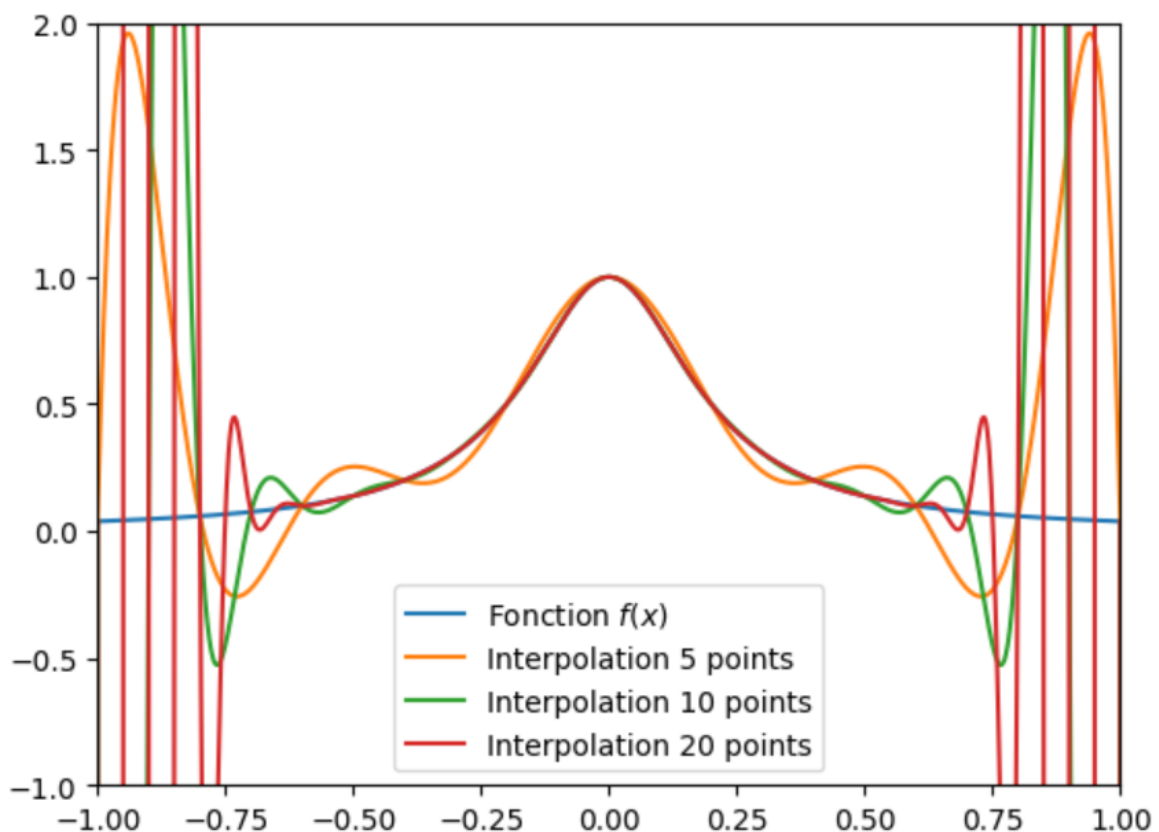
```
Y10 = P10(X)
```

```
Y20 = P20(X)
```

Nous pouvons enfin donner vie à nos courbes, en leur donnant une légende en zoomant sur le graphe, de sorte à ce qu'on ait le meilleur aperçu possible sur ce dernier.

```
plt.plot(X, f(X), label="Fonction $f(x)$")  
plt.plot(X, Y5, label="Interpolation 5 points")  
plt.plot(X, Y10, label="Interpolation 10 points")  
plt.plot(X, Y20, label="Interpolation 20 points")  
  
plt.xlim(-1, 1)  
plt.ylim(-1, 2)  
  
plt.legend()  
plt.show()
```

On obtient le graphe suivant :



On remarque qu'à l'approche de 0, les polynômes d'interpolation approximent plutôt bien la fonction f . Mais sur l'intervalle $[-1 ; -0,75] \cup [0,75 ; 1]$, ces polynômes oscillent grossièrement et n'approximent plus du tout la fonction.

On se propose pour finir de reprendre cette question avec les points de Chebyshev :

$$a_k = \cos\left(\pi \frac{2k+1}{2(n+1)}\right), \quad k = 0, \dots, n.$$

On commence par définir une fonction représentative de la suite ci-dessus :

```
def chebyshev(n):
    return np.array([np.cos(np.pi*(2 * i + 1) / (2 * (n + 1))) for i in range(n + 1)])
```

Puis on répète la même chose que dans l'ancien cas :

```
X = np.linspace(-1, 1, 1000)

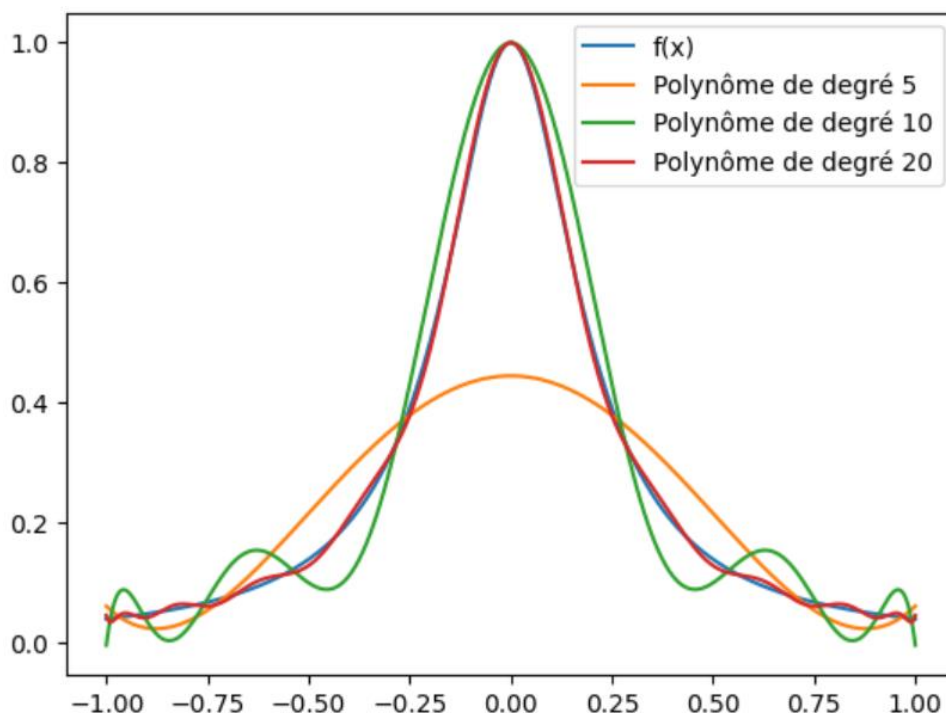
plt.plot(X, f(X), label="f(x)")

X5=chebyshev(5)
X10=chebyshev(10)
X20=chebyshev(20)

Y5=interpolation2(X5,f(X5),X)
Y10=interpolation2(X10,f(X10),X)
Y20=interpolation2(X20,f(X20),X)

plt.plot(X, Y5, label=f"Polynôme de degré 5")
plt.plot(X, Y10, label=f"Polynôme de degré 10")
plt.plot(X, Y20, label=f"Polynôme de degré 20")
plt.legend()
plt.show()
```

On obtient le graphe ci-dessous :



On remarque que l'on n'a plus les grossières oscillations de tout à l'heure et les polynômes interpolateurs approximent plus nettement la fonction f , ce qui prouve la précision et l'utilité des points de Chebyshev.